CYBERSNOVA

CYBER SECURITY INTERNSHIP

PROJECT 4

BASIC WEB VULNERABILITY ASSESSMENT

Vulnerability Assessment Report

Name	OM VYANKAT CHAVAN
Email	Omchavan416@gmail.com
Reference No.	CSN-OC-OL-26294
Registration No.	CSN-OC-2026-294
Domain	Cyber Security
Project Level	Beginner
Project	Basic Web Vulnerability Assessment
Target	http://testasp.vulnweb.com/
Month / Year	October 2026

Description Table

S.No.	Project Description	Page No.
1	Introduction, Objective and Target Information	2
2	Tools Used, Methodology and Commands Used	2–3
3	Summary of Findings and Severity Classification	4
4	Detailed Findings (V-01 to V-09)	4–6
5	Screenshots / Evidence	7–8
6	Security Impact, Recommendations and Conclusion	8–9

1. Introduction

For this project, I carried out a basic security check of the practice website http://testasp.vulnweb.com/. A vulnerability assessment helps identify security weaknesses so they can be fixed before they are misused. This site is intentionally vulnerable and is provided by Acunetix for security training. I focused on basic checks such as identifying the technologies in use, reviewing ports and HTTP headers, and scanning for common server misconfigurations. I did not attempt to exploit any of the issues I observed.

This report describes the tools and steps I used, the issues reported during the assessment, and the actions that could help address them on a real website.

2. Objective

- Identify the technologies and web server used by the target, along with the services visible during the scan.
- Use common security tools to check for possible weaknesses and configuration problems.
- Give each finding a severity rating and explain why that rating is appropriate.
- Recommend practical steps that could reduce or remove the identified risks.
- Record the commands and include evidence from the practical work.

3. Target Information

Target URL	http://testasp.vulnweb.com/
Target Type	Deliberately vulnerable web application built for authorized security training
Resolved IP	44.238.29.244 (AWS EC2, us-west-2)
Web Server	Microsoft IIS 8.5 on Windows
Technology	ASP.NET / classic ASP
Assessment Date	18 October 2026
Scope	Passive and light active checks only: fingerprinting, service detection, header review, Nikto scan and directory brute-forcing. No exploitation, no data extraction, no denial-of-service.

Limitation: This was a beginner-level assessment, so the results are limited to the checks described in this report and the tool output available as evidence. I have not treated a possible issue as confirmed unless the recorded output supported it. This report does not document deeper manual tests for issues such as SQL injection or cross-site scripting using Burp Suite.

4. Tools Used

Tool	Purpose	Use in this project
WhatWeb	Identifies web technologies, headers and cookies.	Quick fingerprint of the target.
curl	Shows the raw HTTP response from a server.	Read response headers with curl -I.
Nmap	Port scanner and service detector.	Find open ports and the service version.
Nikto	Web server vulnerability scanner.	Check for misconfigurations and missing headers.

Tool	Purpose	Use in this project
Gobuster	Directory brute-forcing with a wordlist.	Look for exposed directories and files.

5. Methodology

1. First, I checked that the target was one of the practice sites approved for this exercise.
2. I used WhatWeb to get an initial idea of the website's technology stack.
3. I used curl -I to review the HTTP response headers and check the cookie settings.
4. I used Nmap to check the visible ports and identify the web service and its reported version.
5. I ran Nikto to look for common server configuration issues, missing security headers, enabled HTTP methods and possible information disclosure.
6. I used Gobuster with the Dirb common wordlist to check which directories and files appeared to be accessible.
7. After the checks, I reviewed the results, assigned a severity level to each finding and noted a recommended fix.
8. I included screenshots of the terminal output as supporting evidence.

6. Commands Used

WhatWeb fingerprint

```
whatweb http://testasp.vulnweb.com/
```

HTTP headers

```
curl -I http://testasp.vulnweb.com/
```

Service detection

```
nmap -sV -Pn testasp.vulnweb.com
```

Web server scan

```
nikto -h http://testasp.vulnweb.com/ -o nikto_asp.txt -Format txt
```

Directory enumeration

```
gobuster dir -u http://testasp.vulnweb.com/ -w /usr/share/wordlists/dirb/common.txt -o
gobuster_asp.txt
```

The commands listed here were intended for the authorized practice target. The findings should be read alongside the evidence captured during the practical.

7. Summary of Findings

I grouped the results into four severity levels. High indicates a weakness that could directly put the application or its data at serious risk. Medium is used for issues that could increase the chance or impact of an attack. Low covers issues with a more limited impact on their own, while Informational describes details that are useful to know but are not necessarily vulnerabilities.

ID	Finding	Severity	Found by
V-01	Outdated server and framework (IIS 8.5, ASP.NET 2.0.50727)	Medium	Nmap, Nikto
V-02	No HTTPS, site served over plain HTTP only	Medium	Nmap, curl
V-03	Session cookie set without the HttpOnly flag	Medium	curl, Nikto
V-04	HTTP TRACE method enabled	Medium	Nikto
V-05	Internal IP address leaked in the Location header	Low	Nikto
V-06	Missing anti-clickjacking header (X-Frame-Options)	Low	Nikto
V-07	Missing X-Content-Type-Options header	Low	Nikto
V-08	Server and technology information disclosure	Low	WhatWeb, curl, Nikto
V-09	Exposed directories and robots.txt (/_vti_cnf, /aspnet_client, /cgi-bin)	Low	Gobuster

Nmap also reported that port 80 was open, 961 ports were filtered and 38 were closed. I treated this as informational; these results describe what the scan could see and do not, by themselves, prove that the site is secure.

8. Detailed Findings

V-01: Outdated server and framework

Severity	Medium
Evidence	Nmap: 80/tcp open http Microsoft IIS httpd 8.5. Nikto: x-aspnet-version header 2.0.50727. See Figures 1 to 4.
What I found	The scan output identifies IIS 8.5 and ASP.NET 2.0.50727. These are older technologies, so their support status and patch level should be checked and migrated to currently supported versions where necessary.
Why it matters	Unsupported software may no longer receive security fixes, leaving known weaknesses unaddressed. Exposing version details can also help an attacker identify which issues may be relevant.
How to fix it	Move to supported Windows Server, IIS and .NET versions, and establish a regular patching process.

V-02: No HTTPS

Severity	Medium
Evidence	Nmap shows only port 80 open. WhatWeb, curl and Nikto were all run against http:// and got 200 OK.
What I found	The site responded over plain HTTP, and the scan did not show an open HTTPS service on port 443.

Why it matters	Without encryption, information exchanged with the site could be observed or altered by someone able to monitor the network.
How to fix it	Install and configure a TLS certificate, redirect HTTP traffic to HTTPS, enable HSTS where suitable and set session cookies to Secure.

V-03: Session cookie without HttpOnly

Severity	Medium
Evidence	curl -I shows Set-Cookie: ASPSESSIONIDACASTCSA=...; path=/ with no HttpOnly attribute. Nikto also reports the cookie was created without the httponly flag.
What I found	Because the cookie does not include HttpOnly, JavaScript running in the page may be able to access it.
Why it matters	If a cross-site scripting issue were present, the missing HttpOnly attribute could make cookie theft easier. The missing Secure attribute is also a concern because the site uses HTTP.
How to fix it	Set HttpOnly and Secure on session cookies through the application or an appropriate IIS configuration.

V-04: HTTP TRACE method enabled

Severity	Medium
Evidence	Nikto: OPTIONS shows Allowed HTTP Methods: OPTIONS, TRACE, GET, HEAD, POST.
What I found	The scan reports that TRACE is enabled. This method echoes a request back to the client and is generally unnecessary for a production website.
Why it matters	Leaving unnecessary HTTP methods enabled can increase risk. TRACE should be disabled unless there is a clear, justified need for it.
How to fix it	Disable TRACE through IIS Request Filtering. Review OPTIONS separately and disable it only if the application does not depend on it.

V-05: Internal IP address leaked

Severity	Low
Evidence	Nikto: a request to /aspnet_client returned a Location header containing the internal address 10.0.0.14 (reference CVE-2000-0649).
What I found	The redirect response included a private IP address rather than only the expected public host name.
Why it matters	This information may reveal a small amount about the internal network and could help an attacker with further reconnaissance.
How to fix it	Review the IIS redirect and host-name configuration so that responses do not expose internal IP addresses.

V-06: Missing X-Frame-Options

Severity	Low
Evidence	Nikto: the anti-clickjacking X-Frame-Options header is not present. The curl -I output has no such header either.
What I found	The response does not include the X-Frame-Options header, so this protection against framing is missing.

Why it matters	Without framing protections, a site may be more exposed to clickjacking, where a user is tricked into interacting with content they did not intend to select.
How to fix it	Add X-Frame-Options: SAMEORIGIN or DENY, or configure a Content-Security-Policy frame-ancestors directive.

V-07: Missing X-Content-Type-Options

Severity	Low
Evidence	Nikto: the X-Content-Type-Options header is not set.
What I found	Without this header, browsers may perform MIME-type sniffing instead of consistently respecting the declared content type.
Why it matters	In some situations, MIME sniffing can cause content to be handled in an unsafe way and may increase the impact of other weaknesses.
How to fix it	Set X-Content-Type-Options: nosniff on relevant responses.

V-08: Server and technology information disclosure

Severity	Low
Evidence	curl -I shows Server: Microsoft-IIS/8.5 and X-Powered-By: ASP.NET. Nikto also pulled an x-aspnet-version header from /LklA8o8u.axd.
What I found	The response headers disclose the web server and application technology details.
Why it matters	Version information can help attackers narrow down which known weaknesses to investigate. Limiting unnecessary details reduces this information exposure.
How to fix it	Remove unnecessary X-Powered-By and X-AspNet-Version headers where possible, and reduce the detail exposed in the Server header.

V-09: Exposed directories and robots.txt

Severity	Low
Evidence	Gobuster found /_vti_cnf, /aspnet_client, /cgi-bin (403), /avatars, /html, /images, /jscripts, /templates and /robots.txt (200). See Figure 5.
What I found	Gobuster reported several directory paths, including legacy FrontPage-related paths. The 403 response for /cgi-bin indicates that the path was found but access was denied.
Why it matters	Discoverable paths can reveal details about the website's structure and give someone more locations to investigate. Their presence alone does not prove that the directories contain exploitable vulnerabilities.
How to fix it	Remove unnecessary legacy folders, restrict access to configuration or metadata directories, and review robots.txt to make sure it does not disclose sensitive paths.

9. Screenshots / Evidence

The figures below are included as evidence for the checks described in this report.



Figure 1 – WhatWeb results showing the technologies and headers identified on the target.

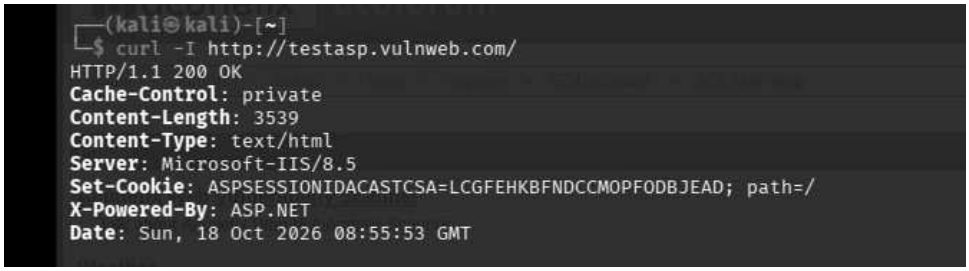


Figure 2 – HTTP response headers reviewed with curl, including the server and session-cookie details.

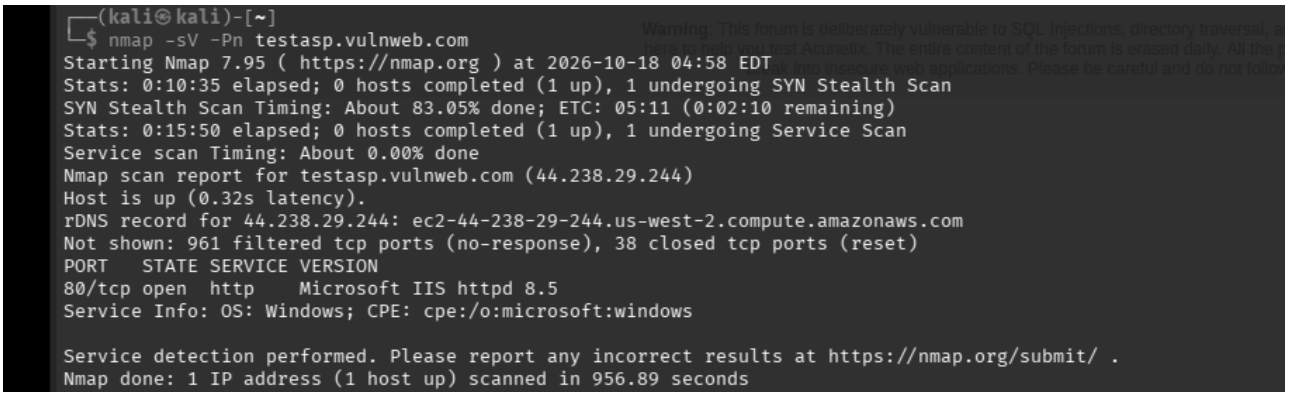


Figure 3 – Nmap output showing the web service detected on port 80.

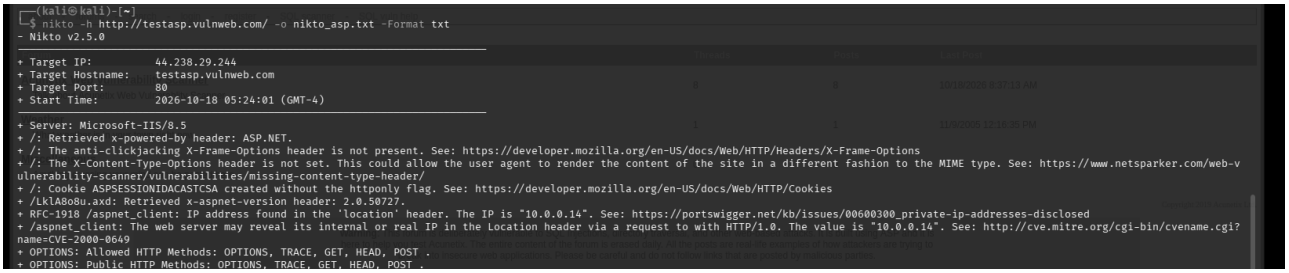


Figure 4 – Nikto results highlighting the configuration issues reported during the scan.

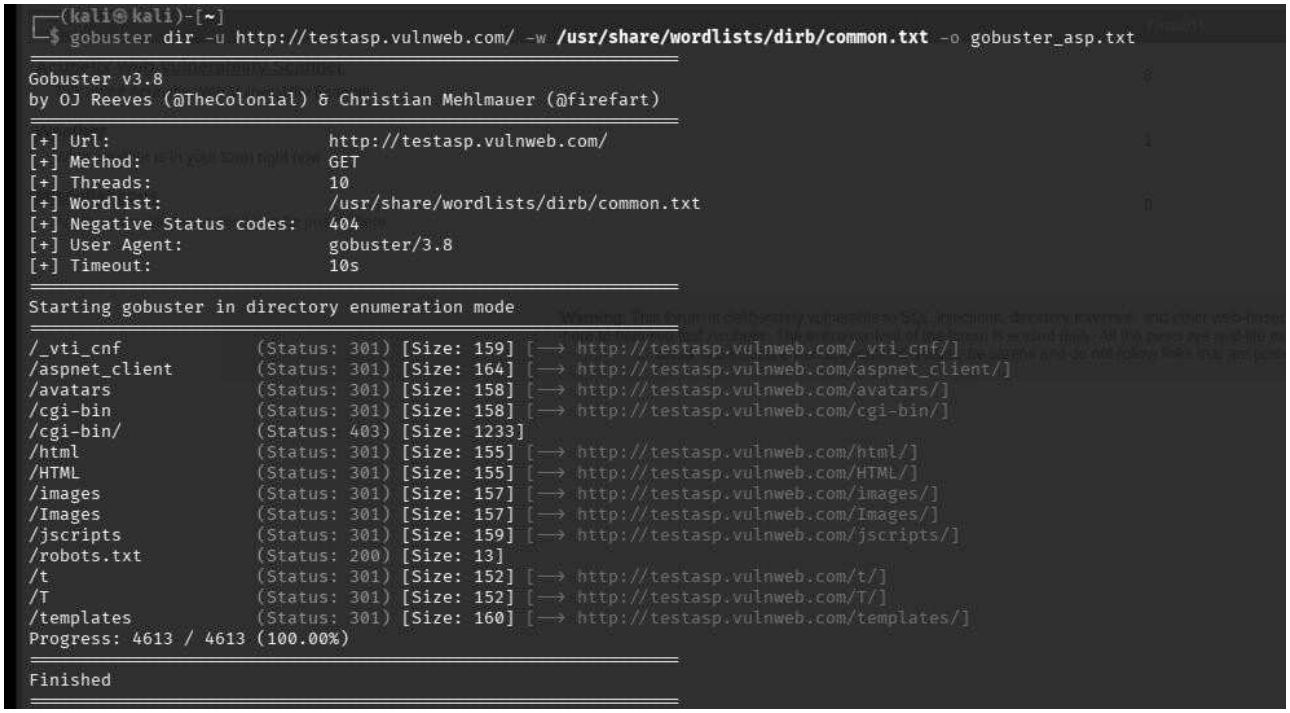


Figure 5 – Gobuster results listing the paths found during directory enumeration.

10. Security Impact

Based on the results recorded here, I did not establish that any single finding would give an attacker complete control of the server. However, several issues appear together: older software is identified by its version, the site responds over unencrypted HTTP, and some recommended browser protections are missing. These weaknesses could make other attacks easier if another vulnerability were present. Overall, I would rate the observed exposure as Medium, with upgrading unsupported software, enabling HTTPS and correcting the session-cookie settings as the main priorities.

The target is intentionally vulnerable and is meant for practice. On a real website, the findings would need to be verified and addressed according to the site's risk and business needs.

11. Recommendations

Fix first (Medium)

- Upgrade IIS, Windows Server and the .NET framework to supported versions, and apply security updates regularly.
- Configure HTTPS across the site, redirect HTTP requests to HTTPS and enable HSTS where appropriate.
- Set the HttpOnly and Secure attributes on session cookies.
- Disable the TRACE method, and disable OPTIONS only if the application does not require it.

Fix next (Low)

- Add the X-Frame-Options and X-Content-Type-Options security headers.
- Reduce unnecessary version details in the Server, X-Powered-By and X-AspNet-Version headers.
- Configure redirects so they do not reveal private internal IP addresses.
- Remove legacy FrontPage folders and unused directories, and restrict access to files that should not be public.

Keep doing

- Repeat authorized vulnerability scans periodically and after major changes.
- If the assessment is extended, carry out careful manual testing with a tool such as Burp Suite for issues like injection and cross-site scripting.

12. Conclusion

This project helped me understand how a basic web vulnerability assessment is carried out and how different tools can be used together. I reviewed the target with WhatWeb, curl, Nmap, Nikto and Gobuster, and the results documented in this report contain nine findings: four rated Medium and five rated Low, plus an informational note about the port scan. The main areas of concern were outdated software, the lack of HTTPS, session-cookie settings, the TRACE method, missing security headers and exposed technology or directory information. I also learned that security problems are not always caused by one major flaw; several smaller configuration issues can combine to increase risk.